

Elevate Labs Internship Submission

SQL Developer Internship • Task 5: Relational Joins

Submitted By:	K. Ameesha	Task Name:	Task 5: SQL Joins
Domain:	SQL Developer	Tools Used:	DB Browser for SQLite / MySQL Workbench

1. Relational Schema & Dataset Initialization

```
-- Initialize Employees Table
CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    EmpName VARCHAR(50),
    DepartmentID INT
);

-- Initialize Departments Table
CREATE TABLE Departments (
    DepartmentID INT PRIMARY KEY,
    DeptName VARCHAR(50)
);

-- Insert Records into Employees
INSERT INTO Employees (EmployeeID, EmpName, DepartmentID) VALUES
(101, 'K. Ameesha', 10),
(102, 'Rohan Verma', 20),
(103, 'Sanya Malhotra', 10),
(104, 'Vikram Rao', NULL);

-- Insert Records into Departments
INSERT INTO Departments (DepartmentID, DeptName) VALUES
(10, 'Data Engineering'),
(20, 'Software Development'),
(30, 'Human Resources');
```

2. SQL Join Executions

A. INNER JOIN

```
-- Queries employees with valid matching departments
SELECT E.EmployeeID, E.EmpName, D.DeptName
FROM Employees E
INNER JOIN Departments D ON E.DepartmentID = D.DepartmentID;
```

B. LEFT JOIN

```
-- Returns all employees regardless of whether a department is assigned
SELECT E.EmployeeID, E.EmpName, D.DeptName
FROM Employees E
LEFT JOIN Departments D ON E.DepartmentID = D.DepartmentID;
```

C. RIGHT JOIN

```
-- Returns all departments regardless of whether employees belong to them
SELECT E.EmployeeID, E.EmpName, D.DeptName
FROM Employees E
RIGHT JOIN Departments D ON E.DepartmentID = D.DepartmentID;
```

D. FULL JOIN

```
-- Full Outer Join using UNION construct for database engine compatibility
SELECT E.EmployeeID, E.EmpName, D.DeptName
FROM Employees E
LEFT JOIN Departments D ON E.DepartmentID = D.DepartmentID
UNION
SELECT E.EmployeeID, E.EmpName, D.DeptName
FROM Employees E
RIGHT JOIN Departments D ON E.DepartmentID = D.DepartmentID;
```

3. Interview Solutions & Technical Analysis

1. Difference between INNER and LEFT JOIN?

INNER JOIN filters out non-matching rows, keeping only intersecting records. LEFT JOIN preserves every record from the left table and populates missing right-table fields with NULL values.

2. What is a FULL OUTER JOIN?

It combines the complete set of records from both tables. Rows without matching records in the counterpart table are padded with NULLs.

3. Can joins be nested?

Yes, queries can nest joins through parentheses ordering or by using derived subqueries within JOIN clauses.

4. How to join more than 2 tables?

Chain sequential JOIN definitions: `SELECT * FROM TableA JOIN TableB ON TableA.id = TableB.id JOIN TableC ON TableB.id = TableC.id;`

5. What is a cross join?

A CROSS JOIN generates a Cartesian product, matching each row of table A with every row of table B without checking join keys.

6. What is a natural join?

A NATURAL JOIN joins tables implicitly on all identical column names present across both relations without needing an explicit ON predicate.

7. Can you join tables without a foreign key?

Yes, foreign key constraints are integrity tools, not join prerequisites. Joins operate on any matching data types or expressions.

8. What is a self-join?

A self-join combines a table with itself using distinct table aliases (e.g., mapping employees to their respective managers within one table).

9. What causes Cartesian product?

Cartesian products occur when join criteria (ON/WHERE clauses) are omitted or when a CROSS JOIN is intentionally executed.

10. How to optimize joins?

Create indexes on join keys, filter input data using subqueries/WHERE prior to joining, pick appropriate join order, and select explicit columns rather than using SELECT *.